

# Applied AI Workshop

A modern, multi-story building with a dark, textured facade and large glass windows. The building is illuminated from within, and the sky above is a vibrant mix of blue, purple, and orange, suggesting a sunset or sunrise. The building's design is contemporary, with a mix of solid and glass panels.

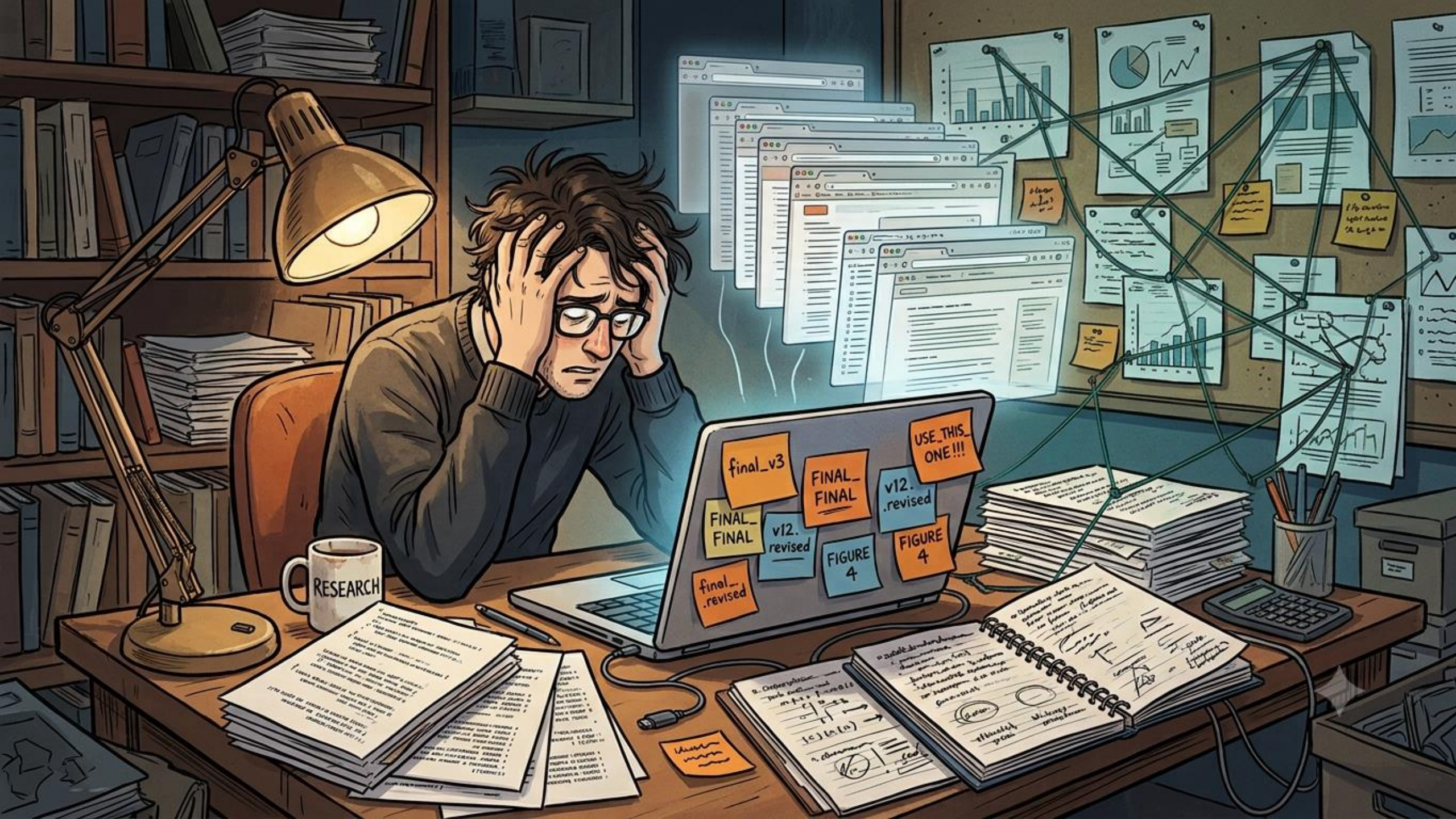
**Git and GitHub**

# A Quick Poll

---







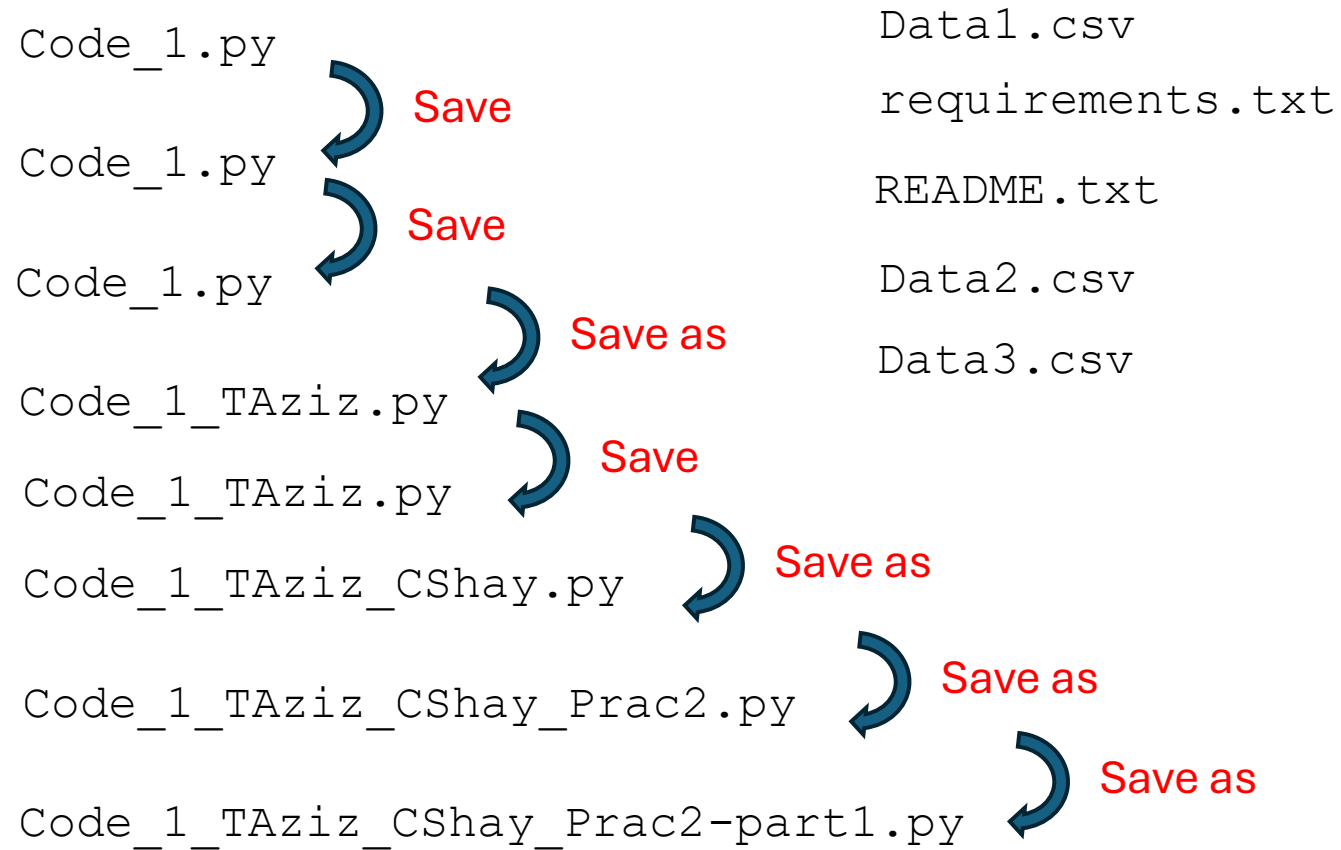






# Typical File Version Management

---



# Git

---



🔍 Type / to search entire site...



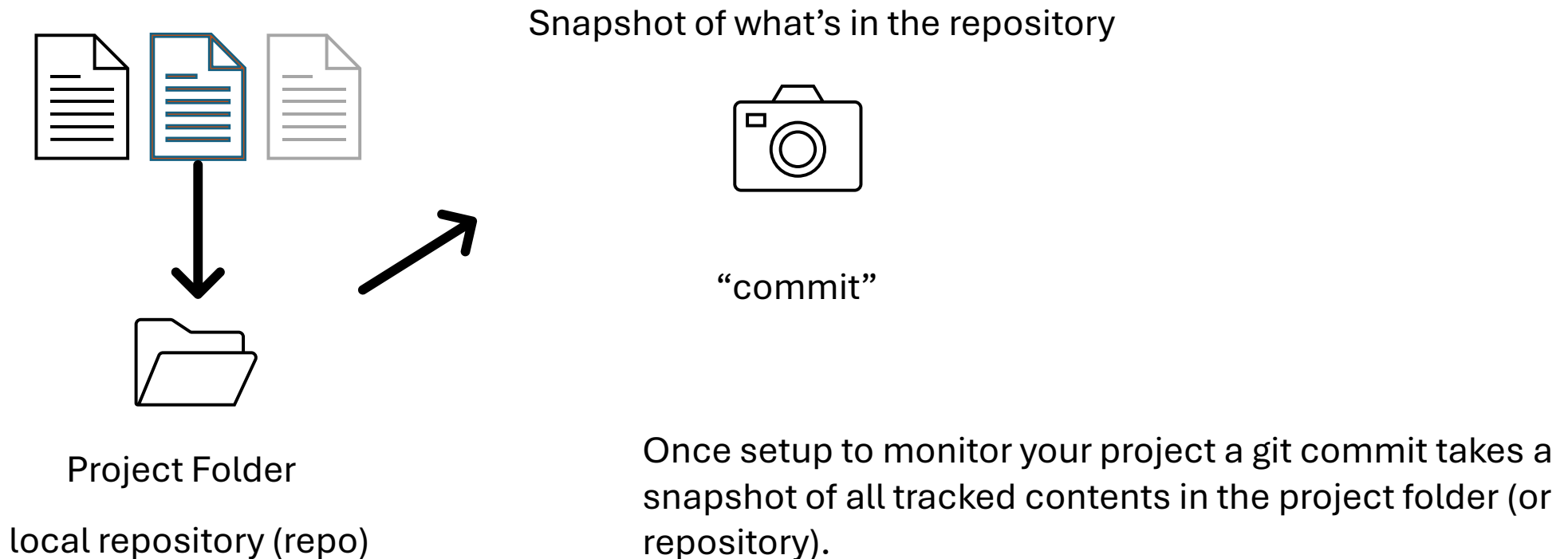
Git is a **free and open source** distributed **version control system** designed to handle everything from small to very large projects with speed and efficiency.

Git is **lightning fast** and has a huge ecosystem of **GUIs**, **hosting services**, and **command-line tools**.



# The basic idea...

---

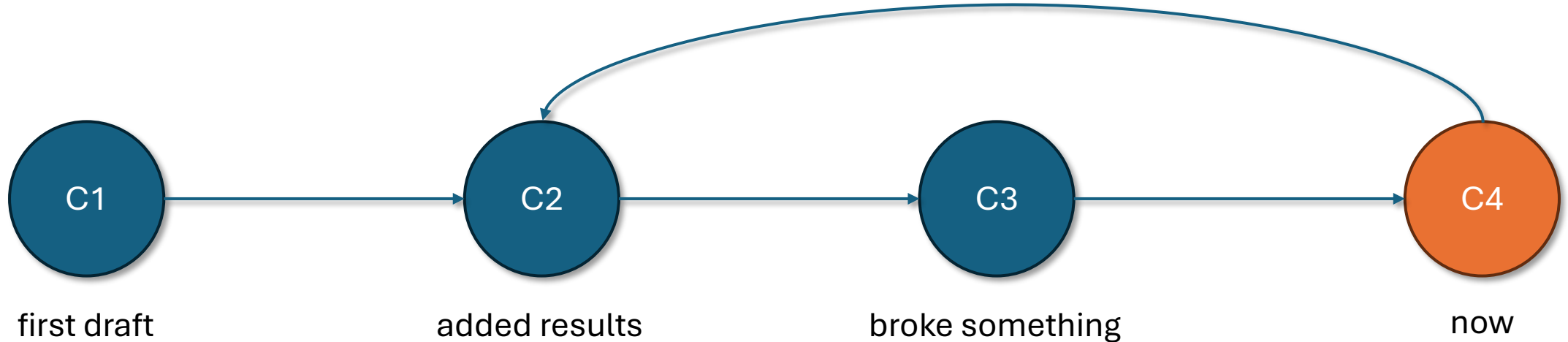


# The Git Way – “commit”

---

commits – C1, C2, C3, C4

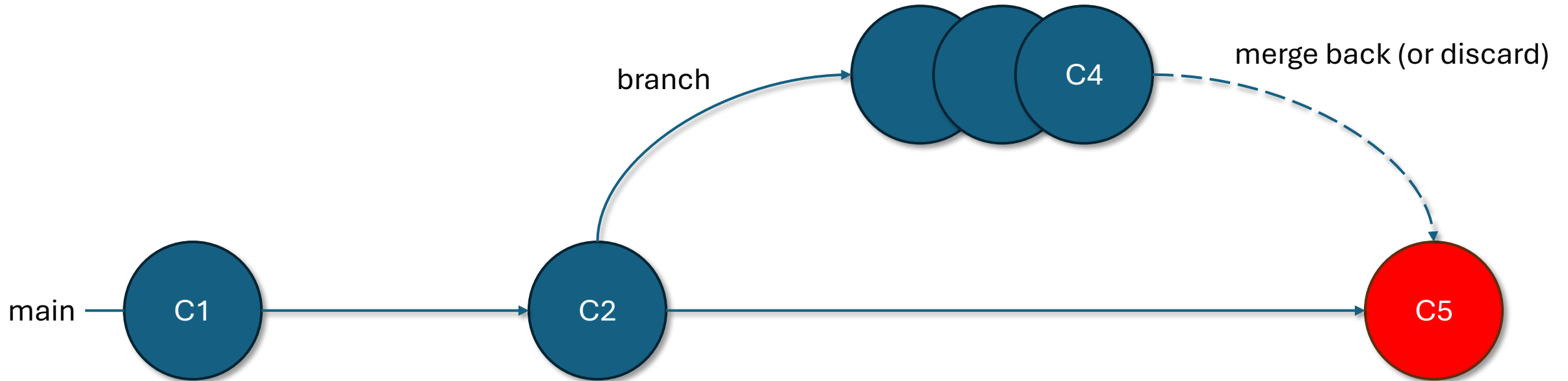
jump back to any snapshot, anytime



Save would have erased C1, C2, C3 – git keeps them all



# A branch is a safe parallel timeline



Your working "main" code stays untouched while the branch runs on its own lane.

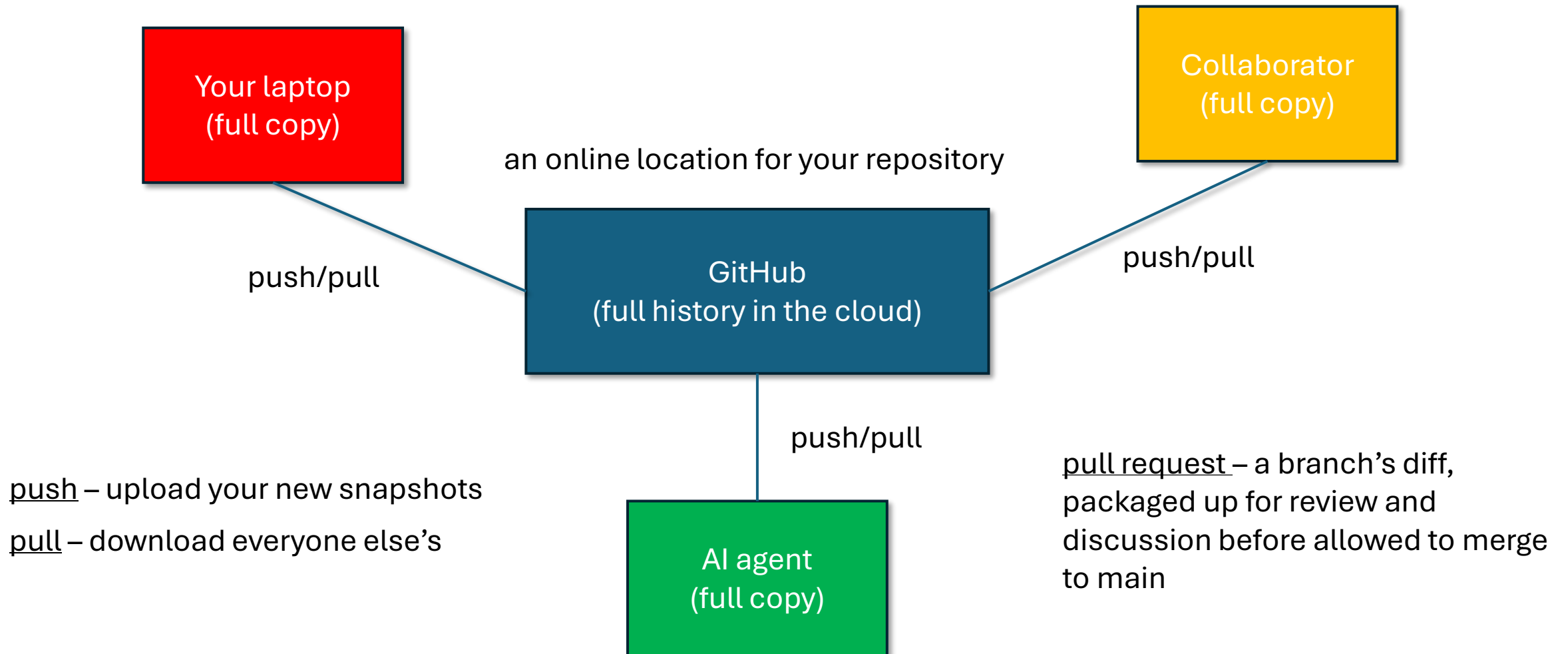
# GitHub

---





# GitHub – the shared hub



# Glossary 1

---

**Git** — A tool that runs on your computer and tracks every version of your project over time. Think of it as a lab notebook for your code: dated, described, permanent entries you can always return to. Git works with or without the internet.

**GitHub** — A website that stores copies of git projects online so you can back them up, share them, and collaborate. Git is the tool; GitHub is a place to keep and share what the tool produces. (Alternatives exist — GitLab, Bitbucket — but GitHub is the most common.)

**Repository (repo)** — A project folder that git is tracking, including its entire history of snapshots. “Cloning a repo” means downloading one; “your repo” is just your tracked project.

**Commit** — One saved snapshot of your whole project at a moment in time, with a short note describing it. Unlike “Save” in Word, a commit doesn’t overwrite the previous version — it adds to a permanent timeline. The single most important word in git.

**Commit message** — The short note attached to a commit (“Add decay table,” “Fix sign error”). Good messages turn your history into a readable logbook.

**Diff** — The line-by-line list of exactly what changed between two snapshots — what was added (usually green) and removed (usually red). This is how you review work, including an AI agent’s, before accepting it.

**Branch** — A separate, parallel line of snapshots that splits off from your main work. You can experiment freely on a branch without touching your working code, then keep it or throw it away. Essential for letting an agent work without risking your good code.

**main** — The default name of the primary branch — your project’s trustworthy, working line. (Older projects may call it master.)

**Merge** — Combining the commits from one branch into another (usually a branch back into main), bringing the new work into your main line.

**Restore / Revert** — Undoing changes. Restore (or “discard”) throws away changes you haven’t committed yet. Revert undoes an earlier commit by adding a new commit on top — safe, because nothing in the history is destroyed.

**Reset** — Moves your branch pointer backward to an earlier commit. Powerful but riskier than revert, because it can discard work. Beginners should prefer revert.

# Glossary 2

---

**Clone** — Download a full copy of a repo from GitHub to your computer (git clone <url>).

**Stage (add)** — Choose which changes go into your next commit (git add). Staging is the “pick what’s in this snapshot” step that happens just before committing. (Forgetting to stage is the #1 beginner stumble — “I committed but my file isn’t there.”)

**Commit** — Save the staged changes as a snapshot (git commit -m "message").

**Push** — Upload your local commits to GitHub (git push).

**Pull** — Download commits from GitHub that you don’t have yet (git pull).

**Status** — Ask git what’s changed and what’s staged right now (git status).

**Log** — View the history of commits (git log, or the Timeline view in VS Code).

**Checkout / Switch** — Move to a different branch or an earlier snapshot.

**Remote** — A copy of your repo that lives somewhere else, usually on GitHub. origin is the default name for your main remote.

**Pull request (PR)** — A proposal on GitHub to merge one branch into another. It packages up the branch’s diff for review and discussion before it’s allowed in. You either approve it or request changes. The core mechanism of collaboration — and of reviewing an agent’s work.

**Fork** — Your own personal copy of someone else’s GitHub repo, so you can work on it independently. (Different from a branch, which lives inside one repo.)

**Issue** — A note on GitHub tracking a bug, task, or idea — like a to-do item or ticket for the project.

**README** — A file (usually README.md) that explains what a project is and how to use it. It’s the first thing shown on a repo’s GitHub page.

**.gitignore** — A file listing things git should never track — datasets, model checkpoints, virtual environments, and secrets like API keys. Set it up before your first commit on a real project.



# Glossary 3

---

**Commit hash** — A unique ID for each commit (like a4f9c2). Lets you point to an exact version — handy for saying “Figure 3 was produced at commit a4f9c2.”

**HEAD** — Git’s way of saying “the snapshot you’re currently sitting on.” Usually the latest commit on your current branch.

**Merge conflict** — What happens when two changes edit the same lines and git can’t decide which to keep. It asks you to choose. Sounds scary, is usually a quick fix — and rare when people work on separate things.

**Markdown (.md)** — A simple way to format plain text (headings, lists, bold) using symbols like # and \*. READMEs and these handouts are written in it.

# Tutorial Videos

---

- [GitHub for Beginners](#)
  - [Git and GitHub Tutorial for Beginners](#)
- (and many, many others)

# For Next Time

---

- Setup Git on your machine and make GitHub accounts
- Create your own repository and make your first commit
- Implement these things with an IDE (like VS Code)
- Clone a project from our organization

Note: For folks doing research at NCSU. Use NCSU GitHub (more security measures)